Complete Agent Generation Guide for LLMs

Purpose

This guide teaches you EXACTLY how to generate a complete Salesforce AI Agentic Function use case. Follow every detail precisely to produce production-ready code.

What You Will Generate

When given a use case description, you will generate **EXACTLY 4 FILES** in a new folder:

1. **[HandlerClassName].apex** - Apex handler class with business logic
2. **[promptname]_PromptCommand.json** - JSON Schema for parameter extraction
3. **AGENT_DESCRIPTION.txt** - Brief agent description for agent record creation
4. **AGENT_SYSTEM_PROMPT.txt** - Complete system prompt defining agent behavior

---

Step-by-Step Process

STEP 1: Analyze the Use Case

Given a use case like: "Update case status to 'In Progress' or 'Closed' with resolution notes"

Extract:

1. **Primary Purpose**: What is the main function? (Update case status)
2. **Salesforce Object**: Which object? (Case)
3. **Operation Type**: CREATE, READ, UPDATE, DELETE (UPDATE)
4. **Required Parameters**: What MUST be provided? (caseId, status)
5. **Optional Parameters**: What is optional? (resolutionNotes)
6. **Business Logic**: Any special rules? (Validate status, prevent reopening closed cases)

STEP 2: Define Naming Convention

Follow this pattern EXACTLY:

Component	Pattern	Example
AI Prompt Name	`operation_Object_by_Identifier`	`update_Case_Status`
Handler Class	`OperationObjectAgenticHandler`	`UpdateCaseStatusAgenticHandler`
Request Param	Same as AI Prompt Name	`update_Case_Status`
Folder Name	Capitalize words with underscores	`Update_Case_Status`

Rules:

- Use camelCase for handler class names
- Use snake_case for prompt names

- Use PascalCase_With_Underscores for folder names
- Operation verbs: create, find, update, delete, search, calculate
- Always include the object name

STEP 3: Create Folder Structure

Create folder: `use-cases/[Use_Case_Name]/`

Example: `use-cases/Update_Case_Status/`

FILE 1: Apex Handler Class

Template Structure

```
/**
 * @description
 *   [ClassName] implements the AIAgenticInterface and serves as a handler
 *   for [purpose]. [What it does in detail].
 *
 * @author              : AI Agentic Architecture
 * @group               : Plumcloud Labs
 * @last modified on    : [MM-DD-YYYY]
 */
public with sharing class [HandlerClassName] implements AIAgenticInterface {

    // SECTION 1: Permission Check Method
    private Boolean hasObjectPerm(String sObjectName, String permType) {
        Schema.DescribeSObjectResult describeResult =
Schema.getGlobalDescribe().get(sObjectName).getDescribe();
        if (permType == 'read')      return describeResult.isAccessible();
        if (permType == 'create')    return describeResult.isCreateable();
        if (permType == 'update')    return describeResult.isUpdateable();
        if (permType == 'delete')    return describeResult.isDeletable();
        return false;
    }

    // SECTION 2: Error Response Methods
    private String errorResponse(Exception ex) {
        return JSON.serialize(new Map<String, Object>{
            'success' => false,
            'status' => 'errored',
            'message' => ex.getMessage(),
            'stackTrace' => ex.getStackTraceString()
        });
    }

    private String errorResponse(String message) {
        return JSON.serialize(new Map<String, Object>{
            'success' => false,
            'status' => 'errored',
            'message' => message
```

```
        });
    }

    // SECTION 3: Request Dispatcher
    public String executeMethod(String requestParam, Map<String, Object>
parameters) {
        try {
            switch on requestParam {
                when '[request_param_name]' {
                    return [methodName](parameters);
                }
                when else {
                    return errorResponse('Method is not defined. This handler only
supports: [request_param_name]');
                }
            }
        } catch (Exception ex) {
            return errorResponse(ex);
        }
    }

    // SECTION 4: Implementation Method
    public String [methodName](Map<String, Object> parameters) {
        // 1. Check permissions
        if (!hasObjectPerm('[ObjectName]', '[operation]')) {
            return errorResponse('Insufficient [operation] permission on
[ObjectName] object.');
        }

        // 2. Validate required parameters
        if (!parameters.containsKey('requiredField') ||
            String.isBlank(String.valueOf(parameters.get('requiredField')))) {
            return errorResponse('requiredField is required.');
        }

        // 3. Extract and validate parameters
        String field1 = String.valueOf(parameters.get('field1'));

        // Additional validation (e.g., ID format, enum values)
        // ...

        // 4. Query if needed (for UPDATE/DELETE operations)
        List<[SObject]> records = [
            SELECT Id, Field1, Field2
            FROM [SObject]
            WHERE Id = :recordId
            LIMIT 1
        ];

        if (records.isEmpty()) {
            return errorResponse('[SObject] not found with ID: ' + recordId);
        }

        [SObject] record = records[0];
```

```
        // 5. Perform DML operation
        try {
            // For CREATE:
            [SObject] newRecord = new [SObject]();
            newRecord.Field1 = field1;
            insert newRecord;

            // For UPDATE:
            record.Field1 = field1;
            update record;

            // For DELETE:
            delete record;

            // For QUERY/READ:
            // Return query results

            // 6. Build redirect URL (for CREATE/UPDATE)
            String redirectUrl = URL.getOrgDomainUrl().toExternalForm() + '/' +
record.Id;

            // 7. Return success response
            return JSON.serialize(new Map<String, Object>{
                'success' => true,
                'status' => 'success',
                'message' => 'Successfully [action] [object]',
                'recordId' => record.Id,
                '[otherRelevantFields]' => 'values',
                'redirectUrl' => redirectUrl,
                'action' => 'redirect'
            });

        } catch (DmlException dmlEx) {
            return JSON.serialize(new Map<String, Object>{
                'success' => false,
                'status' => 'errored',
                'message' => 'DML Error: ' + dmlEx.getDmlMessage(0)
            });
        } catch (Exception ex) {
            return errorResponse(ex);
        }
    }

    // SECTION 5: Helper Methods (if needed)
    private Boolean isValidId(String recordId) {
        if (String.isBlank(recordId)) {
            return false;
        }
        Integer idLength = recordId.length();
        if (idLength != 15 && idLength != 18) {
            return false;
        }
        return recordId.isAlphanumeric();
```

```
    }
}
```

Critical Requirements for Apex Class:

1. **MUST use**: `public with sharing class`
2. **MUST implement**: `AIAgenticInterface`
3. **MUST have**: `hasObjectPerm()` method (exact signature)
4. **MUST have**: Two `errorResponse()` methods (one for Exception, one for String)
5. **MUST have**: `executeMethod()` with switch statement
6. **MUST validate**: All required parameters before DML
7. **MUST check**: Object-level permissions before any operation
8. **MUST handle**: DmlException separately from general Exception
9. **MUST return**: Standardized JSON response format
10. **MUST include**: Redirect URL for CREATE and UPDATE operations

Response Format Standards:

Success Response (CREATE/UPDATE):

```
{
  "success": true,
  "status": "success",
  "message": "Successfully [action] [object]",
  "recordId": "ID",
  "[recordIdentifier]": "Value (e.g., CaseNumber, Name)",
  "[relevantFields]": "values",
  "redirectUrl": "URL",
  "action": "redirect"
}
```

Success Response (READ/QUERY):

```
{
  "success": true,
  "status": "success",
  "message": "Found [X] records",
  "[dataArray]": [...],
  "count": X,
  "[calculatedFields]": "values"
}
```

Error Response:

```
{
  "success": false,
```

```
  "status": "errored",
  "message": "Error description"
}
```

Validation Patterns:

For ID validation:

```
if (!isValidId(recordId) || !recordId.startsWith('[prefix]')) {
    return errorResponse('Invalid [Object] ID format. Must be 15 or 18 characters
starting with [prefix].');
}
```

For enum validation:

```
List<String> validValues = new List<String>{'Value1', 'Value2', 'Value3'};
if (!validValues.contains(fieldValue)) {
    return errorResponse('Invalid value. Valid values: ' +
String.join(validValues, ', '));
}
```

For required field validation:

```
if (!parameters.containsKey('fieldName') ||
    String.isBlank(String.valueOf(parameters.get('fieldName')))) {
    return errorResponse('fieldName is required.');
}
```

FILE 2: Prompt Command JSON Schema

Template Structure

```
{
  "type": "object",
  "required": ["field1", "field2"],
  "properties": {
    "fieldName": {
      "type": "string|number|boolean",
      "maxLength": 255,
      "minimum": 0,
      "maximum": 100,
      "enum": ["value1", "value2"],
      "description": "ONLY the [field] portion. Extract from '[Pattern]: [value]'
or '[Pattern] [value]' patterns. Example: if input is '[Example Input]', extract
```

```
'[Example Output]' only. This field is REQUIRED/OPTIONAL."
    }
  }
}
```

CRITICAL RULES FOR DESCRIPTIONS:

Every field description MUST follow this EXACT format:

1. **Start with**: "ONLY the [field] portion"
2. **Show patterns**: "Extract from '[Pattern]: [value]' or '[Alternative Pattern]' patterns"
3. **Give example**: "Example: if input is '[Input]', extract '[Output]' only"
4. **State requirement**: "This field is REQUIRED" or "This field is OPTIONAL"
5. **Add default if applicable**: "Default: [value] if not specified"

Examples by Field Type:

String Field (ID):

```
"caseId": {
  "type": "string",
  "maxLength": 18,
  "description": "ONLY the Case ID (Salesforce 15 or 18 character ID). Extract
from 'Case ID: [value]' or 'Case: [value]' patterns. Example: if input is 'Update
case 500xx000000ABCD', extract '500xx000000ABCD' only. Case IDs start with '500'.
This field is REQUIRED."
}
```

String Field (Enum):

```
"status": {
  "type": "string",
  "enum": ["New", "Working", "In Progress"],
  "description": "ONLY the status value. Extract from 'Status: [value]' or 'to
[value]' or 'mark as [value]' patterns. Example: if input is 'Update to In
Progress', extract 'In Progress' only. Valid values: 'New', 'Working', 'In
Progress'. This field is REQUIRED."
}
```

Number Field:

```
"minimumProbability": {
  "type": "number",
  "minimum": 0,
  "maximum": 100,
  "description": "ONLY the probability percentage value. Extract from
```

```
'probability: [value]' or 'probability over [value]%' patterns. Example: if input
is 'probability over 70%', extract 70 only. Default: 70 if not specified. This
field is OPTIONAL."
}
```

Boolean Field:

```
"includeClosed": {
  "type": "boolean",
  "description": "ONLY the boolean value. Extract from 'include closed' or 'with
closed' patterns. Example: if input is 'show opportunities including closed',
extract true. Default: false. This field is OPTIONAL."
}
```

Text Field:

```
"resolutionNotes": {
  "type": "string",
  "description": "ONLY the resolution notes or comments. Extract from 'Resolution:
[value]' or 'Notes: [value]' patterns. Example: if input is 'Close with
resolution: Fixed network issue', extract 'Fixed network issue' only. This field
is OPTIONAL."
}
```

Validation Constraints to Include:

- **maxLength**: For string fields (typically 255 for short text, 32000 for long text)
- **minimum/maximum**: For number fields
- **enum**: For picklist/status fields (list ALL valid values)
- **required array**: List all required field names

FILE 3: AGENT_DESCRIPTION.txt

Format:

Simple plain text, 1-3 sentences describing what the agent does for end users.

Template:

```
Create a [agent type] agent that can [list capabilities]. The agent should be
[personality traits] and capable of [key behaviors].
```

Examples:

Update Case Status:

```
Create a case management agent that can update case statuses to various states
including In Progress, Escalated, On Hold, and Closed, with the ability to add
resolution notes when closing cases. The agent should be efficient, detail-
oriented, and ensure proper case status tracking throughout the support lifecycle.
```

Find Opportunities:

```
Create an opportunity intelligence agent that can help sales teams find high-
probability opportunities in their pipeline, analyze deal confidence levels,
calculate total pipeline value for forecasting, and identify which deals are most
likely to close. The agent should be professional, data-driven, and capable of
providing actionable insights and recommendations for deal prioritization.
```

Guidelines:

- Focus on WHAT the agent does for users (not technical implementation)
- Include personality traits (professional, efficient, friendly, data-driven)
- Mention key capabilities
- Keep it business-focused and user-friendly
- 50-150 words maximum

FILE 4: AGENT_SYSTEM_PROMPT.txt

Structure:

The system prompt defines HOW the agent behaves. Use plain text format (NO HTML tags).

Template:

```
You are a [Agent Type] Agent responsible for [primary responsibility].

CORE RESPONSIBILITIES:
- [Responsibility 1]
- [Responsibility 2]
- [Responsibility 3]

COMMUNICATION STYLE:
Your communication should be [style traits], using [type] language appropriate for
[audience].

[CRITICAL SECTIONS BASED ON USE CASE - see examples below]

LIMITATIONS:
```

```
You CANNOT:
- [Limitation 1]
- [Limitation 2]
- [Limitation 3]

If a user requests an action outside your capabilities, politely direct them to
the appropriate agent or suggest they perform the action manually in Salesforce.
```

Critical Sections Based on Operation Type:

For UPDATE/CREATE Operations (requires user input):

Add this section:

```
CRITICAL: [ALWAYS/NEVER RULES]

NEVER claim to have [action] unless you have successfully called the
[function_name] function.

ALWAYS follow this process:
1. Request [Required Info]: If user doesn't provide [info], ask for it
2. Confirm [Details]: Clarify any ambiguous details
3. Execute Function: Call [function_name] with parameters
4. Report Result: Only after success, confirm with details from response

RESPONSE FORMAT BASED ON FUNCTION RESULTS:

ONLY respond with success messages if the function call returns success: true.

- On Success: "[Success message with actual data from response]"
- On Failure: "Unable to [action]: [error message]. Please check [input] and try
again."
- Missing Required Info: "I need [info] to [action]. Please provide [details]."
```

For READ/QUERY Operations:

Add this section:

```
WHEN PRESENTING RESULTS, FOLLOW THIS FORMAT:

1. Summary: "[X] [objects] found with [criteria]"
2. Key Metrics: "[Important numbers/totals]"
3. Top Items: List [3-5] most relevant items
4. Insights: Brief analysis of what the data means
5. Next Steps: Suggest actionable recommendations

KEY METRICS TO ALWAYS INCLUDE:
- [Metric 1]
```

```
- [Metric 2]
- [Metric 3]
```

Example Interactions:

Always include 2-3 example interactions showing:

- What user says
- What agent does (function call)
- What agent responds (based on success/failure)

Format:

```
EXAMPLE INTERACTIONS:

Example 1:
User: "[User input]"
You: [Call function_name with parameters]
If Success: "[Response using actual data from function]"

Example 2:
User: "[User input without required info]"
You: "[Ask for required information]"
```

Full Examples:

For UPDATE operations:

```
You are a Case Management Agent responsible for updating case statuses and adding
resolution notes.

CORE RESPONSIBILITIES:
- Update case statuses to 'New', 'Working', 'In Progress', 'Escalated', 'On Hold',
'Closed', 'Closed - Resolved', and 'Closed - Not Resolved'
- Add detailed resolution notes when closing cases to document the solution
provided
- Ensure proper tracking of case statuses throughout the support lifecycle
- Validate case IDs before making any updates

COMMUNICATION STYLE:
Your communication should be efficient and detail-oriented, using professional
language appropriate for case management.

CRITICAL: ALWAYS ASK FOR CASE ID FIRST

NEVER claim to have updated a case unless you have successfully called the
update_Case_Status function with a valid case ID.

ALWAYS follow this process:
```

```
1. Request Case ID: If the user doesn't provide a case ID, ask for it
2. Confirm Status: If the user doesn't specify the exact status, clarify options
3. Execute Function: Call update_Case_Status with the case ID and status
4. Report Result: Only after the function succeeds, confirm the action with
details from the response

RESPONSE FORMAT BASED ON FUNCTION RESULTS:

ONLY respond with success messages if the function call returns success: true.

- On Success: "Case [CaseNumber] has been successfully updated from '[OldStatus]'
to '[NewStatus]'. View Case: [redirectUrl]"
- On Failure: "Unable to update case: [error message]. Please check the Case ID
and try again."
- Missing Case ID: "I need the Case ID to update the status. Please provide the 15
or 18 character Case ID (starts with '500')."

IMPORTANT GUIDELINES:
- Never fake responses: Do NOT say "the case has been updated" unless the function
returned success
- Always validate inputs: Confirm you have a valid Case ID before attempting any
action
- Be transparent: If you don't have enough information, ask for it
- Use actual data: Use the case number, old status, and new status from the
function response

EXAMPLE INTERACTIONS:

Example 1:
User: "Can you close this case"
You: "I can help you close a case. Please provide the Case ID (a 15 or 18
character ID starting with '500')."

Example 2:
User: "Close case 500xx000000ABCD"
You: [Call update_Case_Status with caseId: "500xx000000ABCD", status: "Closed"]
If Success: "Case 00001234 has been successfully updated from 'In Progress' to
'Closed'. View Case: [URL]"

LIMITATIONS:

You CANNOT:
- Create new cases (use the case creation agent for that)
- Modify case details such as subject, description, priority
- Reopen cases that have been closed
- Delete or merge cases
- Update a case without a Case ID

If a user requests an action outside your capabilities, politely direct them to
the appropriate agent or suggest they perform the action manually in Salesforce.

REMEMBER: NEVER claim an action was completed unless you actually called the
function and it returned success: true.
```

For READ operations:

```
You are an Opportunity Intelligence Agent responsible for helping sales teams find
and analyze high-probability opportunities.

CORE RESPONSIBILITIES:
- Find opportunities based on probability thresholds (default: 70% or higher)
- Calculate total pipeline value for filtered opportunities
- Provide data-driven insights on deal confidence levels and pipeline health
- Identify which deals are most likely to close for prioritization

COMMUNICATION STYLE:
Your communication should be professional and data-driven, using clear business
language appropriate for sales teams.

WHEN PRESENTING OPPORTUNITY RESULTS, FOLLOW THIS FORMAT:

1. Summary: "Found [X] opportunities with [Y]% or higher probability."
2. Total Pipeline Value: "Total potential revenue: [Amount] [Currency]."
3. Top Opportunities: List 3-5 highest value or probability deals.
4. Insights: Provide brief analysis of what the data indicates about pipeline
health.
5. Next Steps: Suggest actionable recommendations for the sales team.

KEY METRICS TO ALWAYS INCLUDE:
- Number of opportunities found
- Total pipeline value (sum of all amounts)
- Average deal size (if multiple opportunities)
- Probability range used for filtering
- Direct links to view opportunities in Salesforce

PROACTIVE RECOMMENDATIONS:

Based on the data you retrieve, provide actionable insights such as:
- Which deals to prioritize for immediate attention
- Pipeline health assessment (strong, needs attention, at risk)
- Suggestions for forecast adjustments

EXAMPLE RESPONSE:

Query: "Show me our best opportunities"

Your Response:

OPPORTUNITY INTELLIGENCE REPORT

Found 8 opportunities with 70% or higher probability

Total Pipeline Value: €1,135,000

TOP OPPORTUNITIES:
- GenePoint - €85,000 (90% probability) - Closes Dec 15, 2025
```

```
- Burlington Textiles - €50,000 (80% probability) - Closes Jan 10, 2026

INSIGHTS:
Your high-confidence pipeline is strong with €1.1M in likely revenue. Focus on the
90% deal for Q4 close.

RECOMMENDED NEXT STEPS:
- Review the GenePoint deal for final blockers
- Schedule close calls for Q4 opportunities

Would you like to adjust the probability threshold?

LIMITATIONS:

You CANNOT:
- Modify opportunity records
- Create new opportunities
- Change probability values or stages
- Predict future probability changes (only report current values)

Always ensure you are providing accurate, current data from Salesforce. The data
is retrieved at the moment of the query.
```

Quality Checklist

Before outputting files, verify:

Apex Handler Class:

- ☑ Uses `public with sharing class`
- ☑ Implements `AIAgenticInterface`
- ☑ Has `hasObjectPerm()` method
- ☑ Has two `errorResponse()` methods
- ☑ Has `executeMethod()` with switch statement
- ☑ Validates all required parameters
- ☑ Checks object-level permissions
- ☑ Handles DmlException separately
- ☑ Returns standardized JSON response
- ☑ Includes redirect URL (for CREATE/UPDATE)

JSON Schema:

- ☑ All required fields in `required` array
- ☑ Descriptions start with "ONLY the [field] portion"
- ☑ Shows extraction patterns
- ☑ Includes concrete examples
- ☑ States REQUIRED or OPTIONAL
- ☑ Has validation constraints (maxLength, enum, min/max)
- ☑ For enum fields, lists ALL valid values

Agent Description:

- ☑ 1-3 sentences
- ☑ User-focused (not technical)
- ☑ Includes personality traits
- ☑ Business-friendly language

Agent System Prompt:

- ☑ Plain text format (no HTML)
- ☑ Defines core responsibilities
- ☑ Specifies communication style
- ☑ Includes critical behavioral rules
- ☑ Has response format guidelines
- ☑ Provides example interactions
- ☑ Lists limitations clearly
- ☑ For UPDATE/CREATE: Emphasizes NOT to fake responses
- ☑ For READ: Includes data presentation format

Common Patterns by Operation Type

CREATE Operations:

- Check `create` permission
- Validate all required fields
- Create new SObject instance
- Set all fields from parameters
- `insert record;`
- Return recordId, recordName, redirectUrl

READ/QUERY Operations:

- Check `read` permission
- Build dynamic SOQL query
- Filter based on parameters
- Calculate aggregates if needed
- Return array of records + metadata (count, totals)

UPDATE Operations:

- Check `read` and `update` permissions
- Query existing record first
- Validate record exists
- Update fields from parameters
- `update record;`
- Return old and new values, redirectUrl

DELETE Operations:

- Check `delete` permission
- Query existing record first
- Validate record exists
- Confirm deletion not prevented by business rules
- `delete record;`
- Return confirmation message

Salesforce Object ID Prefixes

When validating IDs, use these common prefixes:

Object	Prefix
Account	001
Contact	003
Lead	00Q
Opportunity	006
Case	500
Task	00T
Event	00U
Custom Objects	a[0-9][0-9]

Example: Complete Use Case Generation

Input: "Update case status to 'In Progress' or 'Closed' with resolution notes"

Output:

Folder: `use-cases/Update_Case_Status/`

Files:

1. `UpdateCaseStatusAgenticHandler.apex` (175 lines)
2. `update_Case_Status_PromptCommand.json` (30 lines)
3. `AGENT_DESCRIPTION.txt` (3 sentences)
4. `AGENT_SYSTEM_PROMPT.txt` (70 lines)

Naming:

- Prompt Name: `update_Case_Status`
- Handler Class: `UpdateCaseStatusAgenticHandler`
- Method Name: `updateCaseStatus`
- Request Param: `update_Case_Status`

Parameters:

- caseId (string, required) - Salesforce Case ID
- status (string, required, enum) - New status value
- resolutionNotes (string, optional) - Resolution comments

Validations:

- Case ID format (15/18 chars, starts with '500')
- Status value (must be in valid list)
- Case exists
- Case not already closed (if changing to open status)

Response:

- success, status, message, caseId, caseNumber, oldStatus, newStatus, subject, redirectUrl

---

Final Instructions

When you receive a use case:

1. **Analyze** the requirements carefully
2. **Extract** operation type, object, required/optional parameters
3. **Apply naming conventions** exactly as specified
4. **Generate all 4 files** with complete implementation
5. **Validate** against the quality checklist
6. **Output** files in a clear, organized format

Do NOT:

- Skip any sections
- Use placeholder comments like "// Add logic here"
- Generate incomplete code
- Forget error handling
- Omit examples in system prompt
- Use HTML tags in system prompt

DO:

- Follow every pattern exactly
- Include all validation
- Write production-ready code
- Be specific in all descriptions
- Provide concrete examples
- Think through edge cases

Created: December 18, 2025 **Version**: 1.0 **Purpose**: Complete guide for LLMs to generate Salesforce AI Agentic Function use cases